

Async and Await in C# with Examples:

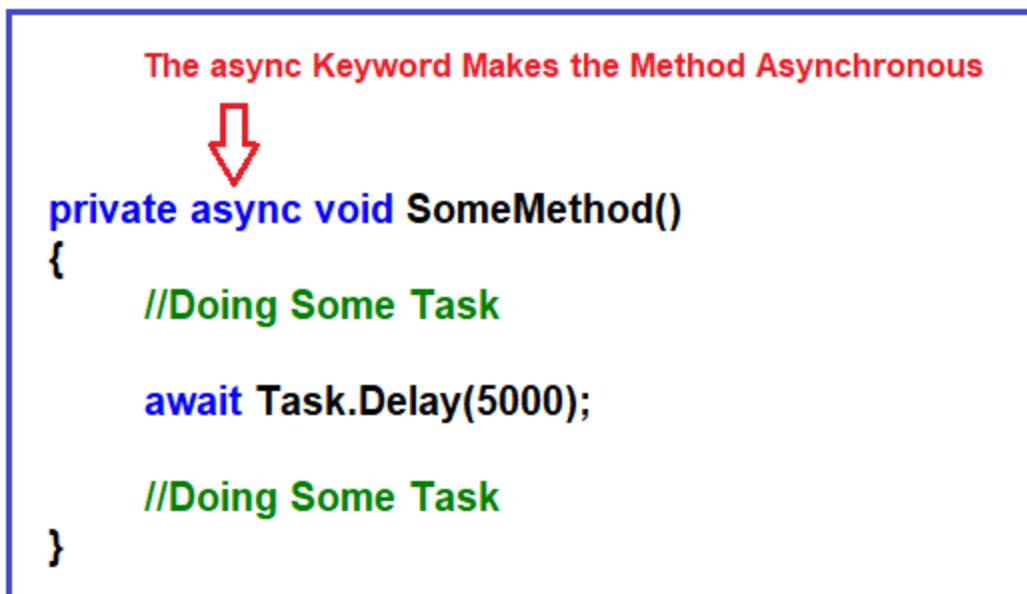
In this article, I am going to discuss how to implement Asynchronous Programming using **Async and Await in C#** with Examples. Please read our previous article where we discussed the basic concepts of [Asynchronous and Parallel Programming](#).

Asynchronous Programming in C#:

Asynchronous programming allows us to have efficient applications where we do not waste resources when they are executed. In this article, we are going to discuss Asynchronous programming. Here, we will look at concepts and patterns for developing effective asynchronous applications. We will start by discussing async, await, and how we avoid freezing the UI. In the next article, we will see the use of Task, which represents a promise of a method of execution that will end in the future. We will talk about how to report Task progress, and how to cancel tasks, and we will also look at some patterns of asynchronous programming.

Async and Await Keyword in C#:

In modern C# code, in order to use asynchronous programming, we need to use async and await keywords. The idea is that if we have a method in which we want to use asynchronous programming, then we need to mark the method with the async keyword as shown in the below image.



For those asynchronous operations for which we do not want to block the execution thread i.e. the current thread, we can use the await operator as shown in the below image.

```
private async void SomeMethod()
{
    //Doing Some Task

    await Task.Delay(5000);

    //Doing Some Task
}
```



The await keyword allows us to suspend the current thread, avoiding blocking it

So, when we use await operator, what we are doing is, we are freeing the current thread from having to wait for the execution of the task. In this way, we are avoiding blocking the current thread that we're using and then that thread can be used in another task.

Async and await works in any .NET development environment like Console applications, Windows Form applications, ASP.NET Core for Web development, Blazor for interactive web applications, etc. Here, we are going to use a Console Application because it is really simple to use. But anything that we do in the Console Application will be applicable to any .NET development environment like ASP.NET Core.

Example to Understand Async and Await in C#:

Please have a look at the below example. It's a very simple example. Inside the main method, first, we print that main method started, then we call the SomeMethod. Inside the SomeMethod, first, we print that SomeMethod started and then the thread execution is sleep for 10. After 10 seconds, it will wake up and execute the other statement inside the SomeMethod method. Then it will come back to the main method, where we called SomeMethod. And finally, it will execute the last print statement inside the main method.

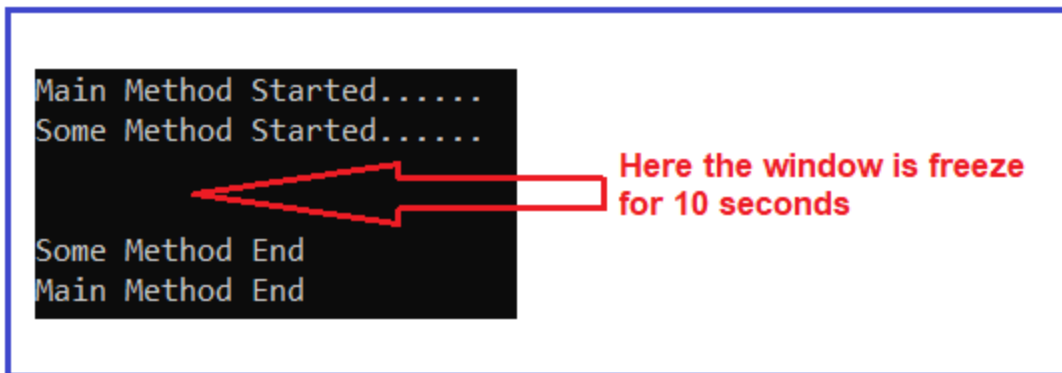
```
using System;
using System.Threading;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            SomeMethod();
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public static void SomeMethod()
        {
            Console.WriteLine("Some Method Started.....");
            Thread.Sleep(TimeSpan.FromSeconds(10));
        }
    }
}
```

```

        Console.WriteLine("\n");
        Console.WriteLine("Some Method End");
    }
}

```

When you execute the above code, you will see that after printing SomeMethod Started....., the console window is frozen for 10 seconds. This is because here we are not using asynchronous programming. One thread i.e. the Main thread is responsible for executing the code And when we call Thread.Sleep method the current thread is blocked for 10 seconds. This is a bad user experience.



Now, let us see how we can overcome this problem by using asynchronous programming. Please have a look at the below image. The **Thread.Sleep()** is a synchronous method. So, we have changed this to **Task.Delay()** which is an asynchronous method. The **Task.Delay()** method exactly does the same thing as **Thread.Sleep()** does.

And, if we want to wait for the task i.e. **Task.Delay** to be done, then we have to use the **await** operator. As we said earlier the **await** operator is going to release the current thread that is running from having to wait for this operation. Therefore, that thread is going to be available for all our tasks. And then after 10 seconds, the thread will be called to the place (i.e. **Task.Delay()**) in order to run the rest code of the **SomeMethod**. As we have used **await** keyword inside the **SomeMethod**, we must have to make the **SomeMethod** as asynchronous as using the **async** keyword.

```

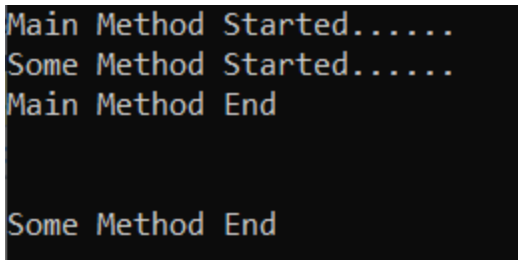
public async static void SomeMethod()
{
    Console.WriteLine("Some Method Started.....");
    await Task.Delay(TimeSpan.FromSeconds(10));
    Console.WriteLine("Some Method End");
}

```

It is important to realize that `await` does not mean that the thread will have to be blocked waiting for the operation. `await` means the thread is free to go to do another thing and then he will come back when this operation (in our example `Task.Delay` i.e. after 10 seconds) is done. The following example code exactly does the same thing.

```
using System;
using System.Threading.Tasks;
namespace AsynchronousProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Started.....");
            SomeMethod();
            Console.WriteLine("Main Method End");
            Console.ReadKey();
        }
        public async static void SomeMethod()
        {
            Console.WriteLine("Some Method Started.....");
            //Thread.Sleep(TimeSpan.FromSeconds(10));
            await Task.Delay(TimeSpan.FromSeconds(10));
            Console.WriteLine("\n");
            Console.WriteLine("Some Method End");
        }
    }
}
```

Output:



```
Main Method Started.....
Some Method Started.....
Main Method End

Some Method End
```

Now, if you run the above code, then you will see that after printing the `Some Method Started` when the statement `Task.Delay()` executed, it will free the current thread, and then that current thread comes and execute the rest of the code inside the main method. And after 10 seconds again thread come back to the `SomeMethod` and execute the rest of the code inside the `SomeMethod`.

So, the bottom line is if you want to have a responsive UI that does not get blocked because of long-running operations, you must use asynchronous programming.

In the next article, I am going to discuss the [Task Class in C#](#) with Examples. Here, in this article, I try to explain how to implement **Asynchronous Programming using Async and Await in C#** with Examples. I hope you enjoy this `Async and Await in C#` with Examples article.